ELEC60004/70068. MACHINE REASONING

1. This question is about search space formalisation and the General Graph Search Engine (GGSE), and 'uninformed' search algorithms.

   a) Explain, with the help of an example, how a state space can be represented (explicitly and implicitly) as a graph, and how an implicit graph representation can be specified in Prolog, so that it can be used for planning and problem-solving with the GGSE.

   [8]

   b) (The Jealous (Heteronormative) Husbands Problem (JHB).) Three husbands with their three wives want to cross a river, using a rowing boat that can carry at most two people. The rowing boat cannot cross the river on its own. Moreover, no woman (wife) can be in the presence of (on the same bank as) another man who is not her husband (i.e., there must either be no men on the same bank, or her husband must be on the same bank).

   Given that we want to know precisely who rows the boat across, specify an appropriate state representation, and justify this state representation with respect to alternative representations for river-crossing problems of this kind.

   [4]

   c) Given this state representation, formulate the state space so that it could be used with the GGSE:

      (*i*) Specify the initial state;
      (*ii*) Specify the final state; and
      (*iii*) Specify the state-change rules.

   Given the symmetry, only the state-change rules for moving one wife, two wives, and one wife plus one husband from the left bank to the right bank need to be specified.

   [6]

   d) Explain how your state-change rules use `findall/3` and backtracking to generate all possible next states.

   [2]

```
/*
** (a)
*/
State Space
-- state: units of information with possible values
-- space: all possible states given all possible combination of values
-- actions: things which change one state into another

Explicit graph G = <N, E, R>
-- N the set of nodes (states)
-- E the set of edges
-- R the incidence relation, i.e. if there is an action that creates an edge between two nodes

Implicit graph G' = <S, Op>
-- S an identified state of N
-- Op the set of operators (state-change rules) which compute all only the members of E

Representation in Prolog:
-- state:compound term encapsulating state information (i.e. value of each unit of information)
-- node:compound term with functor, or tuple, including state and possibly more information,
for example, f-, g- and h-cost, rule used to reach state, etc
-- path: list of nodes
-- graph: list of paths

Example: two water jugs problem

state -- state(Integer1,Integer2)
        where Integer 1 represents water in 5 pint jug, Integer2 represents water in 7 point jug
e.g. initial state is state(0,0)

node -- node( state(0,0), initial )

path -- [ node(state(5,0), fill5), node(state(0,0), initial) ]

graph -- [
          [node(state(5,0), fill5), node(state(0,0), initial)],
          [node(state(0,7), fill7), node(state(0,0), initial)]
         ]

/*
** (b)
*/
** 5-tuple:
        -- list of husbands on left bank
        -- list of wives on left bank
        -- list of husbands on right bank
        -- list of wives on right bank
        -- where bank the boat is on
**
** Because we want to know who rows the boat and who is married to whom they are married
** we have to represent them as individuals with uniquely identifying properties as in
** the bridge-torch, we represent them as individuals with a position on a bank.
** If we did not need to know who rode and marriage relations, they would be fungible
** and husbands and wives could both be represented as an undifferentiated mass, and
** the constraint would simply be there should not be more wives than husbands on
** either bank (as in the missionaries and cannibals problem).
**
*/

/*
** (c)
*/
```

```
** Initial State
([h1,h2,h3], [w1,w2,w3], [], [], left )

** Final State
([], [], _, _, right)

** State change rules
** Only moving left to right is required, for 1 husband, 2 husbands, or
** (one husband and one wife.
** This is the complete solution, i.e. why only 3 state-change rules are required :)

state_change( move1, (ML,WL,MR,WR,Bank), (NewML,NewWL,NewMR,NewWR,OppositeBank) ) :-
        move_one( Bank, OppositeBank, ML, WL, MR, WR, NewML, NewWL, NewMR, NewWR ),
        valid_state( NewML, NewWL, NewMR, NewWR ).

state_change( move2, (ML,WL,MR,WR,Bank), (NewML,NewWL,NewMR,NewWR,OppositeBank) ) :-
        move_two( Bank, OppositeBank, ML, WL, MR, WR, NewML, NewWL, NewMR, NewWR ),
        valid_state( NewML, NewWL, NewMR, NewWR ).

state_change( move11, (ML,WL,MR,WR,Bank), (NewML,NewWL,NewMR,NewWR,OppositeBank) ) :-
        move_couple( Bank, OppositeBank, ML, WL, MR, WR, NewML, NewWL, NewMR, NewWR ),
        valid_state( NewML, NewWL, NewMR, NewWR ).

%%% valid state
%%%
valid_state( NewML, NewWL, NewMR, NewWR ) :-
        for_each_wife( NewWL, NewML ),
        for_each_wife( NewWR, NewMR ).

for_each_wife( [], _ ).

for_each_wife( [Wife|RestW], Husbands ) :-
        wife_ok( Wife, Husbands ),
        for_each_wife( RestW, Husbands ).

wife_ok( _, [] ).
wife_ok( W, Husbands ) :-
        member( H, Husbands ),
        married( H, W ).

married( h1, w1 ).
married( h2, w2 ).
married( h3, w3 ).


%%% move one husband l->r
%%%
move_one( left, right, ML, WL, MR, WR, NewML, WL, [One|MR], WR ) :-
        append( Front, [One|Back], ML ),
        append( Front, Back, NewML ).

%%% move one wife l->r
%%%
move_one( left, right, ML, WL, MR, WR, ML, NewWL, MR, [One|WR] ) :-
        append( Front, [One|Back], WL ),
        append( Front, Back, NewWL ).

%%% move one husband r->l
%%%
move_one( right, left, ML, WL, MR, WR, [One|ML], WL, NewMR, WR ) :-
        append( Front, [One|Back], MR ),
        append( Front, Back, NewMR ).

%%% move one wife r->l
```

```
%%%
move_one( right, left, ML, WL, MR, WR, ML, [One|WL], MR, NewWR ) :-
        append( Front, [One|Back], WR ),
        append( Front, Back, NewWR ).

%%% move two husbands l->r
%%%
move_two( left, right, ML, WL, MR, WR, NewML, WL, [One, Two|MR], WR ) :-
        append( Front1, [One|Back1], ML ),
        append( Front2, [Two|Back2], Back1 ),
        append( Front1, Front2, Temp ),
        append( Temp, Back2, NewML ).

%%% move two wives l->r
%%%
move_two( left, right, ML, WL, MR, WR, ML, NewWL, MR, [One, Two|WR] ) :-
        append( Front1, [One|Back1], WL ),
        append( Front2, [Two|Back2], Back1 ),
        append( Front1, Front2, Temp ),
        append( Temp, Back2, NewWL ).

%%% move two husbands r->l
%%%
move_two( right, left, ML, WL, MR, WR, [One, Two|ML], WL, NewMR, WR ) :-
        append( Front1, [One|Back1], MR ),
        append( Front2, [Two|Back2], Back1 ),
        append( Front1, Front2, Temp ),
        append( Temp, Back2, NewMR ).

%%% move two wives r->l
%%%
move_two( right, left, ML, WL, MR, WR, ML, [One, Two|WL], MR, NewWR ) :-
        append( Front1, [One|Back1], WR ),
        append( Front2, [Two|Back2], Back1 ),
        append( Front1, Front2, Temp ),
        append( Temp, Back2, NewWR ).

%%% move one of each l->r
%%%
move_couple( right, left, ML, WL, MR, WR, NewML, NewWL, [OneM|MR], [OneW|WR] ) :-
        append( Front1, [OneM|Back1], ML ),
        append( Front1, Back1, NewML ),
        append( Front2, [OneW|Back2], WL ),
        append( Front2, Back2, NewWL ).

%%% move one of each r->l
%%%
move_couple( right, left, ML, WL, MR, WR, [OneM|ML], [OneW|WL], NewMR, NewWR ) :-
        append( Front1, [OneM|Back1], MR ),
        append( Front1, Back1, NewMR ),
        append( Front2, [OneW|Back2], WR ),
        append( Front2, Back2, NewWR ).

%%% opposite_bank
%%%
opposite_bank( left, right ).
opposite_bank( right, left ).

/*
** (d)
*/
Using append as a relation rather than I/O function, i.e. in move_one, using it 'backwards'
to breakdown the input list and generate one person on the bank at a time, and the others
on backtracking. Note this solution then uses it 'forwards' to reconstruct the bank without
the one we have selected from the fronts and backs of the list generated from 'backwards'
```

2. This question is about search space formalisation and the General Graph Search Engine (GGSE), and 'informed' search algorithms.

a) Explain the difference between the Best First and A^* search algorithms.

[2]

b) Explain how the A^* search algorithm expands the paths in a graph defined as $G' = \langle S, op \rangle$.

Prove that the solution path returned by the A^* search algorithm is optimal.

[4]

c) In the context of A^* search, explain the conditions that need to be satisfied for $h(G)$, if G is a goal state, and for h to be monotonic and admissible.

Explain why these conditions might make one heuristic perform "better" than another.

Comment on this explanation with respect to optimality and complexity of the A^* search algorithm.

[6]

d) In the 8-Puzzle, aka the *ah*-Puzzle, the tile locations are labelled $[0..8]$, with locations $0,1,2$ labelling the top row, $3,4,5$ the middle row, etc. The blank can then 'move' (i.e., swap places with) any tile that is in a location connected to the blank's location: for example, if the blank is in location 0, then it can swap with whatever is in location 1 or location 3, The problem state can then represented by a list of 9 integers, the first integer represents the location of the blank tile, the second integer the location of the *a* tile, etc.

A good heuristic is to count Manhattan distance; and an even better heuristic is to count the Manhattan distance plus the number of adjacent reversals (i.e. swapping two adjacent tiles that are in each other's locations), times two.

Explain why this is an admissible heuristic.

Specify a predicate `ith/3` that for an integer index $i, i \geq 0$ and a list, 'returns' the element at the *i*th position in the list, i.e.:

```
| ?- ith( 0, [a,b,c], E ).

E = a;

no

| ?- ith( 3, [a,b,c], E ).

no
```

Hence, or otherwise, specify a predicate that counts the number of adjacent reversals (times two) in a list representing a state of the *ah*-Puzzle.

(You do not need to specify predicates for counting Manhattan Distance.)

[8]

/* ** (a) */

Best first: uses a heuristic to estimate h-cost of getting from frontier node to goal; chooses path with frontier node having least h-cost

A*: uses a cost function to accumulate the actual cost (g-cost) of getting from the start node to the frontier node; uses a heuristic to estimate h-cost of getting from frontier node to goal; sums the g-cost+h-cost to give the f-cost which is the cheapest estimated cost of a path from the start node to a goal node with this frontier node on the path. A* then choose the path with the frontier node having the least g-cost.

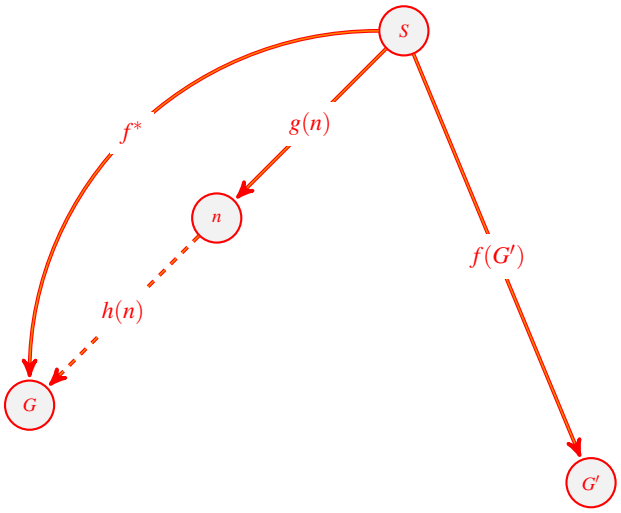
/* ** (b) */

Expands that path in whichever P_i has been partially or completely filled in, whose frontier node's h-cost is least

- Let G be an optimal goal state, with path cost f^*
- Let G" be a sub-optimal goal state with path cost $f(G')$. Then

$$f(G') = g(g') + h(G') = g(G') + 0 = g(G')$$

- Suppose A* returns G' as its (sub-optimal) solution, so $f(G') > f^*$, so $g(G') > f^*$.
- Now consider a node n on the frontier node on a path leading to G



$f^* \geq f(n)$	h is admissible and the path cost is monotonically increasing
$f(n) \geq f(G')$	otherwise n would have been selected for expansion before G'
$f^* \geq f(G')$	transitivity
$f^* \geq g(G')$	

- This is a contradiction
- So either G' was optimal, or A* never expands a sub-optimal goal

/* ** (c) */

Cost: If G is a goal state, h(G) = 0

Monotonic: path cost always increasing, or made monotonic by pathmax function

Admissible: never overestimates actual cost of getting to goal

- Expanding the node with the lowest f-cost
  - If we estimate $h_o > h_{actual}$, so $g(n)+h_o > g(n')+h(n')$, then we might find a path from S to a sub-optimal goal G' through n'
  - If we estimate $h_u < h_{actual}$, at some point $g(n)+h_u < g(n')+h(n')$, so n will be expanded
  - If we always under-estimate, at some point we will expand the solution path ending in G before any solution path ending in G'

- Imagine drawing a histogram with increasing f-cost along the x-axis
  - Then map all the nodes onto the histogram
  - A* will expand all those nodes to the left of the f^* bar
  - Therefore the 'trick' is to get as many nodes to the right of the f^* bar ...
  - ... without ever over-estimating the h-cost

- Make $h(n)$ as large as possible (without over-estimating)
  - A* expands all nodes for which $f(n) < f^*$
  - So A* expands all nodes for which $h(n) < f^* - g(n)$
  - Therefore if $h_2(n) \geq h_1(n)$, for all n, h_2 may expand fewer nodes

Doesn't matter which heuristic, A* is still optimal

Big-O does worst case complexity, actual complexity significantly reduced by concise and accurate knowledge representation, and efficient computation (as close as possible to actual cost without ever over-estimating) with tractable algorithm (i.e. don't forget the complexity of computing the heuristic).

```
/*
** (d)
*/
ith( 0, [E|_], E ).
ith( I1, [_|T], E ) :-
        I is I1 - 1,
        ith( I, T, E ).

adjacent_reversal( List, Sum ) :-
        findall( (X,Y),
                (connected(X,Y),
                 ith(X, List, Y),
                 ith(Y, List, X) ),
                DR ),
        length( DR, Sum ).

connected( 0, 1 ).
connected( 0, 3 ).
connected( 1, 0 ).
connected( 1, 4 ).
connected( 1, 2 ).
etc.
```

3. This question is about knowledge representation and reasoning with logic and Prolog.

a) In the context of Machine Reasoning, define *resolution*, and state the resolution rule.

[4]

b) In the context of Machine Reasoning, define *unification*. Explain why unification is relevant to resolution, and give a unification algorithm.

[4]

c) Consider the following English statements:

Everyone who has ball-control skills and team-working skills is a footballer.
Everyone who is fit and fast is an athlete.
Everyone who is an athlete has physique.
Everyone who is a footballer has technique.
Everyone who has physique and technique is a goal-machine.
Bryan is fit and fast, and has ball-control skills and team-working skills.

Express these statements in first-order predicate logic, and convert them to a set of Horn clauses.

[4]

d) Prove, using resolution and proof by negation, that *Bryan is a goal-machine*. The proof should show all the unifiers.

[4]

e) Explain what it means for an algorithm for deciding the satisfiability of a logical formula to be *sound* and *complete*.

[4]

/* ** (a) */

Definition: Resolution is a single valid inference rule that, in propositional logic, produces a new formula implied by two formulas containing complementary literals. With an appropriate procedure, the resolution rule yields a sound and complete algorithm for deciding the satisfiability of a propositional formula.

Rule

$$\frac{p \vee q \quad \neg q}{p}$$

Generally

$$\frac{\neg p_1 \vee \neg p2 \vee \ldots \vee \neg p_{i-1} \vee \neg p_i \vee \neg p_{i+1} \vee \ldots \vee \neg p_n \quad \neg q_1 \vee \neg q_2 \vee \ldots \neg q_m \vee p_i}{\neg p_1 \vee \neg p2 \vee \ldots \vee \neg p_{i-1} \vee \neg q_1 \vee \neg q_2 \vee \ldots \neg q_m \vee \neg p_{i+1} \vee \ldots \neg p_n}$$

Closure (contradiction)

$$\frac{p \quad \neg p}{\bot}$$

For predicate logic, two literals p and $\neg p$ can be resolved with the most general unifier ϕ.

/* ** (b) */

Definition

Unification is a process of attempting to identify two symbolic expressions by the matching of terms and the replacement of certain sub-expressions (variables) by other expressions. The **unification problem** for two expressions is finding a way of replacing their variables with other terms such that the two resulting terms are syntactically identical.

Algorithm

- If S and T are constants, then they unify if and only if they are identical
- if S is a variable and T is any term, then they unify with the substitution $\{S \mapsto T\}$
  - This includes the case that T is also a variable
- if both S and T are compound terms, then they unify if and only if:
  - They have the same functor name
  - They are of the same arity
  - All of their pairwise corresponding arguments unify

/* ** (c) */

$$\forall x.(ball_control_skills(x) \wedge team_working_skills(x)) \rightarrow footballer(x)$$
$$\forall x.(fit(x) \wedge fast(x)) \rightarrow athlete(x)$$
$$\forall x.athlete(x) \rightarrow physique(x)$$
$$\forall x.footballer(x) \rightarrow technique(x)$$
$$\forall x.(physique(x) \wedge technique(x)) \rightarrow goal_machine(x)$$
$$ball_control_skills(bryan) \wedge team_working_skills(bryan) \wedge fit(bryan) \wedge fast(bryan)$$

1	$\neg ball_control_skills(X1) \vee \neg team_working_skills(X1) \vee footballer(X1)$		
2	$\neg fit(X2) \vee \neg fast(X2) \vee athlete(X2)$		
3	$\neg athlete(X3) \vee physique(X3)$		
4	$\neg footballer(X4) \vee technique(X4)$		
5	$\neg physique(X5) \vee \neg technique(X5) \vee goal_machine(X5)$		
6	$ball_control_skills(bryan)$		
7	$team_working_skills(bryan)$		
8	$fit(bryan)$		
9	$fast(bryan)$		

/* ** (d) */

0	$\neg goal_machine(bryan)$		negated conclusion
10	$\neg physique(bryan) \vee \neg technique(bryan)$	$\{X5 \rightsquigarrow bryan\}$	0, 5
11	$\neg athlete(bryan) \vee \neg technique(bryan)$	$\{X3 \rightsquigarrow bryan\}$	3, 10
12	$\neg fit(bryan) \vee \neg fast(bryan) \vee \neg technique(bryan)$	$\{X2 \rightsquigarrow bryan\}$	2, 11
13	$\neg fast(bryan) \vee \neg technique(bryan)$		8, 12
14	$\neg technique(bryan)$		9, 13
15	$\neg footballer(bryan)$	$\{X4 \rightsquigarrow bryan\}$	4, 14
16	$\neg ball_control_skills(bryan) \vee \neg team_working_skills(bryan)$	$\{X1 \rightsquigarrow bryan\}$	1, 15
17	$\neg team_working_skills(bryan)$		6, 16
18	$\bot$		7, 17

/* ** (d) */

Sound

If the proof procedure proves something, then it is a tautology

Complete

if there is a tautology, then the proof procedure proves it

4. This question is about reasoning in Propositional Logic, Predicate Logic, and Propositional Modal Logic, using the **KE** calculus.

Important: any answer to any part that doesn't use line numbers and rule references to indicate how new formulas have been derived will not get full marks.

a) Prove, using the **KE** calculus for Propositional Logic, that the following statements are true:

(*i*) $\{a, (a \wedge b) \rightarrow c, \neg c, \neg b \rightarrow (d \wedge e)\} \vdash \neg b \wedge (d \wedge e)$
(*ii*) $\{e \rightarrow g, (e \rightarrow g) \rightarrow (g \rightarrow f), e\} \vdash f$
(*iii*) $\{\neg(r \rightarrow m), (r \wedge e) \rightarrow \neg t, e\} \vdash \neg(t \vee m)$
(*iv*) $\{a \rightarrow b, b \rightarrow a, e\} \vdash (a \leftrightarrow b) \wedge e$

[8]

b) Translate the following syllogism into Predicate Logic. Use **KE** to prove that the conclusion follows from the premises: *All owners of stately homes are country gentlemen. Boris is the owner of a stately home. Therefore Boris is a country gentleman.*

Translate the following syllogism into Predicate Logic. Use **KE** to build a model which makes the premises true and the conclusion false: *Some knowledge is power. Some power is measured in Watts. Therefore some knowledge is measured in Watts.*

[6]

c) Using the **KE** calculus for Propositional Modal Logic, prove that the characteristic formulas for reflexivity, symmetry and transitivity relations are theorems of logic **S5**, but not of logic **K**:

(*i*) $\Box P \rightarrow P$
(*ii*) $P \rightarrow \Box\Diamond P$
(*iii*) $\Box P \rightarrow \Box\Box P$

[6]

```
/*
** (a)
*/
(i)

| ?- ke( [a, (a&b)->c, -c, -b->(d&e), -(-b & d & e )] ).
   1     a
   2     a & b -> c
   3     - c
   4     - b -> d & e
   5     - ((- b) & d & e)
   6     beta  2,3       - (a & b)
   7     beta  6,1       - b
   8     beta  4,7       d & e
   9     alpha 8         d
  10     alpha 8         e
  11     beta  5,7       - (d & e)
  12     beta  11,9      - e
         close 12,10

yes

(ii)

| ?- ke( [e->g,(e->g)->(g->f),e,-f] ).
   1     e -> g
   2     (e -> g) -> g -> f
   3     e
   4     - f
   5     beta  1,3       g
   6     beta  2,1       g -> f
   7     beta  6,5       f
         close 4,7

yes

(iii)

| ?- ke( [-(r->m),(r&e)-> -t, e, -(-(t+m))] ).
   1     - (r -> m)
   2     r & e -> - t
   3     e
   4     - (- (t + m))
   5     alpha 1         r
   6     alpha 1         - m
   7     dne   4         t + m
   8     beta  7,6       t
   9     beta  2,8       - (r & e)
  10     beta  9,5       - e
         close 10,3
```

```
yes


(iv)

| ?- ke( [a->b,b->a,e,-((a<->b)&e)] ).
   1     a -> b
   2     b -> a
   3     e
   4     - ((a <-> b) & e)
   5     beta  4,3       - (a <-> b)
   6     br-1  1               - a
   7           beta  2,6       - b
   8           eta   5,6       b
               close 8,7
   9     br-2  1               a
  10           beta  1,9       b
  11           eta   5,9       - b
               close 11,10

yes
```

/* ** (b) */

In Predicate Logic

1	$\forall x.owner_stately_home(x) \rightarrow country_gentleman(x)$	premise
2	$owner_stately_home(boris)$	premise
3	$\neg(country_gentleman(boris))$	neg-conc
4	$owner_stately_home(boris) \rightarrow country_gentleman(boris)$	$\forall$, 1
5	$country_gentleman(boris)$	β, 4, 2
	close (3,5)	

1	$\exists x.knowledge(x) \wedge power(x)$	premise
2	$\exists x.power(x) \wedge watts(x)$	premise
3	$\neg(\exists x.knowledge(x) \wedge watts(x))$	$\neg$ conc
4	$knowledge(k1) \wedge power(k1)$	$\exists$, 1
5	$knowledge(k1)$	α, 4
6	$power(k1)$	α, 4
7	$power(k2) \wedge watts(k2)$	$\exists$, 2
8	$power(k2)$	α, 7
9	$watts(k2)$	α, 7
10	$\neg(knowledge(k1) \wedge watts(k1))$	$\forall$, 3
11	$\neg(knowledge(k2) \wedge watts(k2))$	$\forall$, 3
12	$\neg watts(k1)$	β, 10, 5
13	$\neg knowledge(k2)$	β, 11, 9
open		

Build a model which makes the premises true, and the conclusion false.

So $k1$ is knowledge and a power, but not a watts; and $k2$ is a power and a watts, but not a knowledge. Hence I can build a model in which the premises are true, but there isn't a thing which is a knowledge and a watts.

/* ** (c) */

In S5: we just label the world with integers, we can move to any world that is available on the branch

(i)

1	[1]:	$\neg(\Box p \to p)$	$\neg conc$
2	[1]:	$\Box p$	$\alpha, 1$
3	[1]:	$\neg p$	$\alpha, 1$
4	[1];	p	$\Box$
		close (3,4)	

(ii)

1	[1]:	$\neg(p \to \Box\Diamond p)$	$\neg conc$
2	[1]:	p	$\alpha, 1$
3	[1]:	$\neg(\Box\Diamond p)$	$\alpha, 1$
4	[2];	$\neg(\Diamond p)$	$\Diamond$
5	[1]:	$\neg p$	$\Box$
		close (2,5)	

(iii)

1	[1]:	$\neg(\Box p \to \Box\Box p)$	$\neg conc$
2	[1]:	$\Box p$	$\alpha, 1$
3	[1]:	$\neg(\Box\Box p)$	$\alpha, 1$
4	[2];	$\neg(\Box p)$	$\Diamond$
5	[3]:	$\neg p$	$\Diamond$
6	[3]:	p	$\Box$
		close (5,6)	

In K, we label the worlds with sequences of integers such that it the sequence captures the accessibility relation. In **K**, σn is accessible from σ. Thus:

(i)

1	[1]:	$\neg(\Box p \to p)$	$\neg conc$
2	[1]:	$\Box p$	$\alpha, 1$
3	[1]:	$\neg p$	$\alpha, 1$
		open	

(ii)

1	[1]:	$\neg(p \to \Box\Diamond p)$	$\neg conc$
2	[1]:	p	$\alpha, 1$
3	[1]:	$\neg(\Box\Diamond p)$	$\alpha, 1$
4	[1,1];	$\neg(\Diamond p)$	$\Diamond$
		open	

(iii)

1	[1]:	$\neg(\Box p \to \Box\Box p)$	$\neg conc$
2	[1]:	$\Box p$	$\alpha, 1$
3	[1]:	$\neg(\Box\Box p)$	$\alpha, 1$
4	[1,1];	$\neg(\Box p)$	$\Diamond$
5	[1,1,1]:	$\neg p$	$\Diamond$
6	[1,1]:	p	$\Box$
		open	

5. This question is about formulating a state space for 2-player games using the General Graph Search Engine (GGSE), and the Simplified Event Calculus (EC).

One variant of the Game of Nim is played with a pile of matches, and two players *red* and *blue* who take it in turns to remove 1, 2 or 3 matches from the pile. The winner is the player to take the last match(es).

a) Making some reasonable assumptions about which player takes the first turn and the initial number of matches in the pile, formulate the state space so that it could be used with the GGSE:

(*i*) Define a state representation;
(*ii*) Specify the initial state;
(*iii*) Specify the final state; and
(*iv*) Specify the state-change rules.

[8]

b) Making some reasonable assumptions about which player takes the first turn and the initial number of matches in the pile, specify the Game of Nim as `initially/1` facts about identified multi-valued fluents.

Then define `initiates/3` axioms of the Simplified EC for reasoning about the effects of any action or actions played in the game on these fluents.

These axioms should ensure that a move is a valid move (i.e., the same player cannot take consecutive turns, cannot take more than three matches, cannot take a negative number of matches, etc.).

[8]

c) Explain, using the Game of Nim as an example, how knowledge is represented and reasoned with in the Simplified Event Calculus compared to the state-space formulation as a graph, and briefly discuss the implications for planning to solve problems of this kind.

[4]

```
/*
** (a)
*/
(i)
State representation: 3-tuple:
        -- whose turn : {red, blue}
        -- number of matches in pile : integer
        -- winner : {red, blue}

(ii)
Initial State, assume red goes first, 21 matches
%%% Note we don't need to have a separate argument for the Winner, as the player
%%% to take the last match is the winner. However, this is quite neat because it
%%% means that every node in the path will be instantiated to the winner, so we
%%% determine the winner by inspecting any node on the solution path. We can also
%%% change the winning conditions more easily, e.g if last player to take a match was
%%% the loser
(red, 21, Winner)

(iii)
Goal State
(Winner, 0, Winner)

(iv)
state_change( take, state(Current, Pile, Winner), state(Next, SmallerPile, Winner) ) :-
        take_matches( N ),
        SmallerPile is Pile - N,
        SmallerPile >= 0,
        next_player( Current, Next ).

take_matches( 1 ).
take_matches( 2 ).
take_matches( 3 ).

next_player( red, blue ).
next_player( blue, red ).

/*
** (b)
*/
initially( matches=21 ).                %%% assume arbitrary number of matches in the pile
initially( player=red ).                %%% assume red goes first
initially( winner=none ).               %%% no winner to begin with

%%% take matches from the pile, initiate fewer matches
%%%
initiates( take( Player, Matches ), pile=Less, T ) :-
        valid_turn( Player, Matches, T, More ),                 %%% what it says on the tin
        Less is More - Matches.                                 %%% simple math for new number of matches

%%% toggle whose turn
%%%
initiates( take( Player, Matches ), player=Opponent, T ) :-
        valid_turn( Player, Matches, T, _ ),                    %%% what it says on the tin
        opponent( Player, Opponent ).                           %%% initiate opponent's turn

%%% winner check
%%%
initiates( take( Player, Matches ), winner=player, T ) :-
        valid_turn( Player, Matches, T, _ ),                    %%% what it says on the tin
        holdsAt( pile=Matches, T ).                             %%% taking the last match(es)
```

```
valid_turn( Player, Matches, T, More ) :-
        Matches > 0, Matches < 4,                          %%% valid number of matches
        holdsAt( player=Player, T ),                       %%% really is this player's turn
        holdsAt( pile=More, T ),                           %%% current number of matches
        More >= Matches.                                   %%% can't leave negative pile

opponent( red, blue ).
opponent( blue, red ).

/*
** (c)
*/
With state space formulation, we have a single global state, whereas in the EC,
each fluent represents a local state. We can reconstruct the global state by asking
what \texttt{holdsAt} any time point.

Then to apply a state-change rules, we have to explicitly list everything in the state,
including those things that are not changed. With EC, we only have to specify the axioms
for those fluents whose values are changed by the corresponding event, everything else
stays the same (common sense law of inertia).

This can be a limitation as the state-space scales, we have to track more and more parameters
as information in the state space gets larger and larger. The limitation of the EC is the amount
of recursion that can cause problems
as the narrative gets longer, and for example there is no caching going on.
```